PROJECT REPORT

On

Bank Management System using C

Submitted in partial fulfilment of the course

Programming in C [24CS001]

BATCH-2025

Submitted to:

Faculty Name: Mr. Rinku Saini

Submitted by:

Divyam
2511981080

Department of Computer Science & Engineering

CHITKARA UNIVERSITY, HIMACHAL PRADESH

1. Introduction

In the modern era, banking systems play a crucial role in managing financial transactions efficiently. Traditionally, banking operations such as maintaining customer records, updating balances, and tracking transactions were performed manually. This manual approach is not only time-consuming but also increases the chances of human error, data redundancy, and record mismanagement. As the number of customers and transactions grows, managing data manually becomes increasingly difficult and unreliable.

With the advancement of technology, computerized systems have replaced traditional methods, providing faster, more secure, and accurate solutions. A Bank Management System is an application that allows users to perform basic banking operations digitally, ensuring proper data management and easy accessibility.

The purpose of this project is to design and implement a simple Bank Management System using the C programming language. This system uses file handling techniques to store and retrieve data, ensuring persistence even after the program is closed. It enables users to create accounts, deposit and withdraw money, check balances, and manage customer records efficiently. The project helps in understanding real-world applications of programming concepts such as structures, functions, and file operations.

2. Problem Statement

In traditional banking systems, most of the data is maintained manually in registers or files, which leads to several issues. These include difficulty in searching records, high chances of human error, data duplication, and risk of data loss due to physical damage or misplacement. Updating customer information and transaction records manually is also a time-consuming process.

Moreover, manual systems lack security and efficiency, making it difficult to handle a large number of customers and transactions. There is no proper mechanism to quickly retrieve or update account details. These limitations highlight the need for a computerized banking system.

Therefore, the problem is to develop an efficient and reliable Bank Management System that can automate banking operations, reduce manual effort, and ensure accuracy and security of data.

3. Objective

The main objectives of this project are:

- To design and develop a user-friendly Bank Management System using C programming
- To implement file handling techniques for storing and managing account data
- To provide functionalities such as account creation, deposit, withdrawal, and balance inquiry
- To ensure data accuracy and reduce manual errors in record management
- To develop a menu-driven program for better user interaction
- To apply concepts of structures and functions for modular programming
- To simulate real-life banking operations in a simplified manner
- To enhance programming and problem-solving skills through practical implementation

4. System Requirements

Software Requirements:

- C Programming Language
- C Compiler (GCC, Turbo C, or Visual Studio Code)
- Operating System (Windows, Linux, or macOS)

Hardware Requirements:

- A computer system with minimum 4GB RAM
- Processor with basic computing capability
- Keyboard and display unit
- Storage space for saving files

5. Methodology

A. Working / Algorithm

The Bank Management System follows a structured approach to perform different operations. The step-by-step working of the system is as follows:

Step 1: Start the program

Step 2: Display a menu with available options such as Create Account, Deposit, Withdraw, Check Balance, Search Account, Delete Account, and Exit

Step 3: Accept the user's choice using input

Step 4: Based on the selected option, call the respective function

Step 5: For account creation, take user details and store them in a file using structures

Step 6: For deposit and withdrawal, read account details, update balance, and save changes back to the file

Step 7: For searching and checking balance, retrieve and display the required account information

Step 8: For deletion, remove the selected account safely using file handling techniques

Step 9: Repeat the process until the user selects the Exit option

Step 10: Terminate the program

B. Working Explanation

The system is designed as a menu-driven program that interacts with the user through a console interface. When the program starts, it displays a list of operations that the user can perform. The user selects an option, and the program executes the corresponding functionality.

Each operation is handled by a separate function, ensuring modularity and easy maintenance of the code. The system uses structures to store account information such as account number, name, and balance. All data is stored in files using file handling functions, which ensures that the data is not lost when the program is closed.

During deposit and withdrawal operations, the system reads the account details from the file, updates the balance, and writes the updated data back to the file. It also maintains a transaction record for each operation. For searching and displaying account details, the system scans the file and retrieves the required information.

The deletion operation is performed carefully by copying all records except the one to be deleted into a temporary file and then replacing the original file. This ensures data safety and integrity.

6. Tools and Concepts Used

This project is implemented using the C programming language and incorporates various important programming concepts:

- **Structures:** Used to define and store account details such as account number, name, and balance
- **File Handling:** Functions like fopen(), fclose(), fwrite(), fread(), fprintf(), and fscanf() are used to store and retrieve data from files
- **Functions:** Used to divide the program into smaller modules for better readability and maintainability
- **Conditional Statements:** Used for decision-making (if-else conditions)
- **Loops:** Used to repeat operations and implement menu-driven functionality
- **Switch-Case:** Used to handle multiple user choices efficiently
- **Basic Input/Output Operations:** Used for user interaction

7. Source Code (Major Modules/Functions)

The project consists of several important functions, each responsible for a specific operation:

- **createAccount()** → This function is used to create a new account by taking user input and storing it in a file
- **depositMoney()** → This function allows the user to deposit money into an existing account and updates the balance
- **withdrawMoney()** → This function enables withdrawal of money after checking if sufficient balance is available
- **checkBalance()** → This function displays the details and current balance of a specific account
- **searchAccount()** → This function searches for an account using the account number and displays the details if found
- **deleteAccount()** → This function deletes an account safely from the file using a temporary file approach

8. Results (Screenshots)

The Bank Management System was tested successfully with different inputs, and the following results were obtained:

- Screenshot 1: Menu-driven interface displaying all options
- Screenshot 2: Successful creation of a new account
- Screenshot 3: Deposit operation showing updated balance
- Screenshot 4: Withdrawal operation with balance validation
- Screenshot 5: Displaying account details using search functionality
- Screenshot 6: Deletion of an account and updated records

The system performed all operations accurately and efficiently, demonstrating correct implementation of file handling and program logic.

9. Conclusion

The Bank Management System developed in C successfully demonstrates how basic banking operations can be automated using programming concepts. The project provided practical exposure to file handling, structures, and modular programming. It also helped in understanding how real-world systems are designed and implemented.

The system is capable of performing essential operations such as account creation, deposit, withdrawal, and record management efficiently. It reduces manual effort, improves accuracy, and ensures better organization of data. All the objectives defined at the beginning of the project were successfully achieved.

Overall, this project enhanced problem-solving skills and provided valuable experience in developing a real-life application using the C programming language.

10. Future Scope

Although the current system performs basic banking operations effectively, it can be further enhanced with additional features:

- Development of a graphical user interface (GUI) for better user experience
- Implementation of user authentication and password protection for security
- Addition of advanced features such as loan management and interest calculation
- Integration with a database system for better data management
- Conversion into a web-based or mobile application
- Implementation of multi-user access and networking capabilities

These improvements can make the system more advanced, secure, and suitable for real-world applications.